BitPub Documento dei Requisiti

1. Introduzione e Descrizione

Cos'è BitPub BitPub è una piattaforma software progettata nell'ambito dell'Internet of Things (IoT), con lo scopo di connettere a Internet giochi fisici tradizionali come calciobalilla, freccette e biliardo. Il sistema permette di raccogliere e analizzare in modo centralizzato i dati sullo svolgimento delle partite, mantenendo inalterata la natura fisica del gioco.

L'architettura centralizza e gestisce due livelli fondamentali:

1. **Livello Edge (Locale):** Un dispositivo hardware situato nel locale che coordina i giochi, raccoglie dati dai sensori (o simulatori software) tramite protocollo MQTT in LAN e gestisce un'interfaccia utente locale per punteggi e tabelloni.
2. **Livello Cloud (Server Centrale):** Il componente principale che mantiene tutte le informazioni su utenti, locali, giochi e tornei. Espone interfacce REST puramente stateless per fornire statistiche e servizi avanzati agli utenti tramite API.

L'obiettivo ingegneristico principale è garantire la tolleranza ai guasti di rete: il sistema garantisce il funzionamento offline con memorizzazione temporanea e sincronizzazione differita verso il Cloud non appena la connessione viene ripristinata.

2. Requisiti funzionali di alto livello

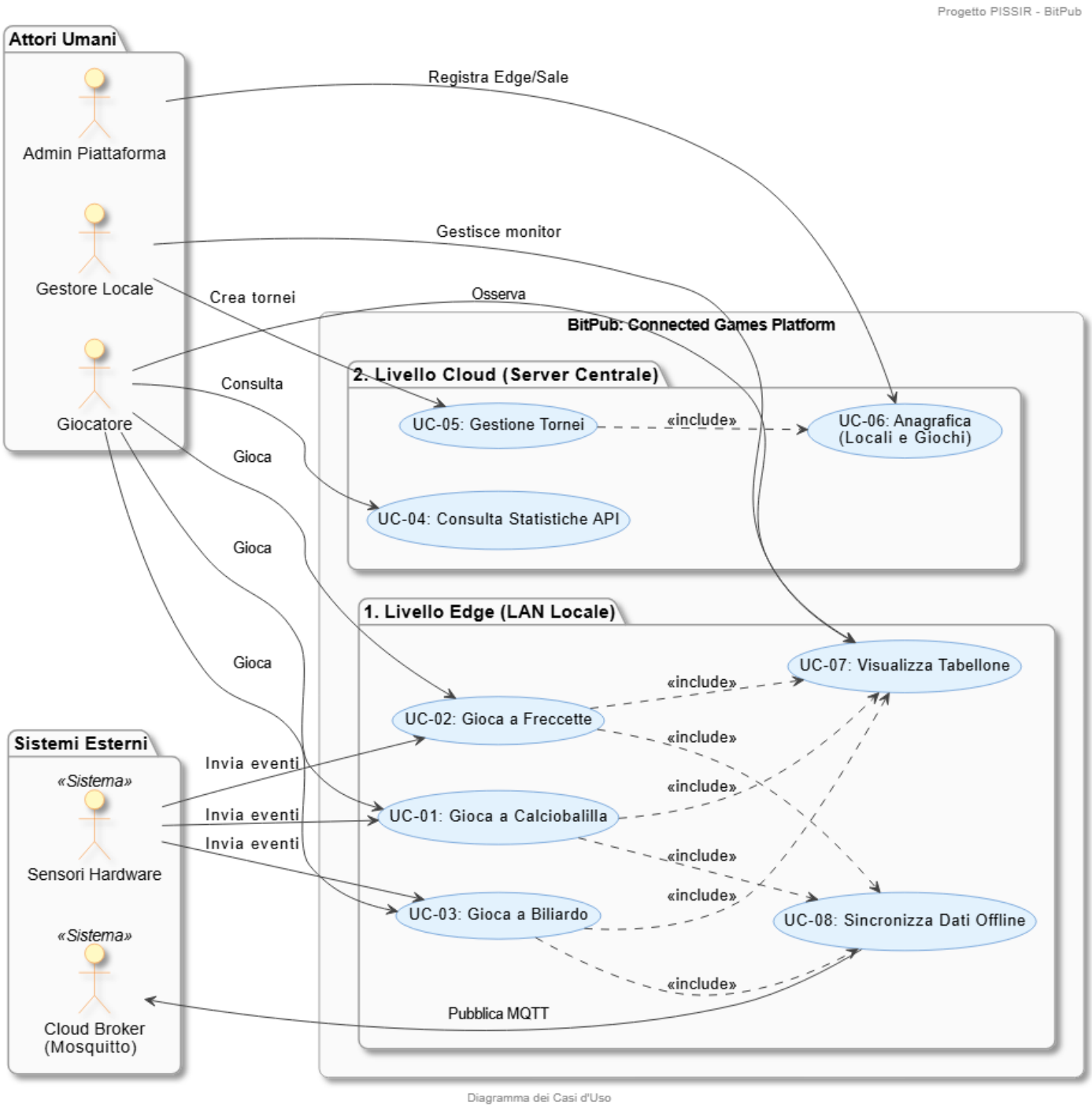
ID	Requisito (Alto Livello)	Descrizione e Dettagli
RF-01	Acquisizione Dati Sensoriali	Il componente Edge deve raccogliere tempestivamente i dati dai sensori (o simulatori multithread) installati sui giochi presenti nel locale tramite MQTT locale.

ID	Requisito (Alto Livello)	Descrizione e Dettagli
RF-02	Sincronizzazione Offline/Online	L'Edge deve memorizzare temporaneamente le partite se disconnesso, accodando i JSON in strutture concorrenti (synchronized), e sincronizzarle con il server centrale al ripristino della rete.
RF-03	Esposizione API REST	Il server centrale deve fornire interfacce REST state-less, basate su URL a sostantivi, Semantic Versioning e principi HATEOAS, per permettere l'accesso ai dati archiviati (utenti, locali, partite).
RF-04	Gestione Tornei e Squadre	Il sistema deve permettere la creazione di competizioni per un singolo tipo di gioco, supportando partite individuali o registrazioni di squadre e generando classifiche finali.
RF-05	Elaborazione Statistiche	Calcolo e fornitura di dati analitici globali come partite giocate, win-rate, classifiche giocatori e utilizzo dei giochi nei vari locali.
RF-06	Interfaccia Utente Locale	Gestione tramite l'Edge di display sui giochi fisici (sviluppata nativamente in JavaFX) per mostrare punteggi e timer in tempo reale.
RF-07	Gestione Anagrafiche	Il server centrale (gestito dagli Amministratori di Piattaforma) deve gestire in modo esclusivo il database globale di utenti, locali e dispositivi Edge associati.

3. Requisiti non funzionali

Categoria	Requisito (Descrizione)	Criterio / Vincolo
Resilienza e Affidabilità	Il sistema deve garantire l'assenza della perdita di dati durante i periodi di disconnessione dei Locali.	*Vincolo:* Implementazione del QoS 1/2 e della *Durable Session* (setCleanSession(false)) sul client MQTT Eclipse Paho dell'Edge Node. Tasso di perdita dati tollerato: 0%.
Concorrenza	Gestione sicura del multithreading nel Cloud e nell'Edge Node per evitare corruzione dei dati durante la raccolta simultanea da più sensori.	*Criterio:* Utilizzo di blocchi synchronized per l'accesso allo stato condiviso. Il database PostGres SQL deve essere protetto da Data Access Object (DAO) thread-safe.
Sicurezza	La comunicazione in uscita dall'Edge verso il Cloud Broker MQTT (porta 8883) deve essere crittografata.	*Vincolo:* Implementazione di certificati TLS/OpenSSL custom sul server Mosquitto (require_certificate true).
Usabilità API	Le API devono permettere una navigazione dinamica da parte del client senza hardcoding degli URL.	*Vincolo:* Adozione rigorosa dell'architettura HATEOAS (Hypermedia As The Engine Of Application State) nei payload JSON di risposta.

4. Diagramma dei casi d'uso



5. Descrizione dei casi d’uso

Caso d'uso: Elaborazione Eventi Calciobalilla

Dettaglio del caso d’uso 1	Elaborazione e invio punteggio partita Calciobalilla
Id:	UC-01
Responsabile:	Stefano
Attori principali:	Simulatore Sensori
Attori secondari:	Edge Node, Server Cloud
Precondizioni:	1. Il simulatore hardware del Calciobalilla è in esecuzione in LAN. 2. L'Edge Node è connesso ed è sottoscritto al topic MQTT corretto (es. locali/+/calciobalilla/+/eventi).
Flusso principale:	1. Il simulatore rileva un evento fisico e pubblica un JSON ("Gol Rosso" o "Gol Blu") sul broker locale. 2. L'Edge Node riceve il messaggio tramite la libreria Eclipse Paho. 3. L'Edge aggiorna il modello dati della partita in locale in modo thread-safe (synchronized). 4. L'Edge aggiorna immediatamente l'interfaccia UI locale (JavaFX) per mostrare il nuovo punteggio invocando Platform.runLater(). 5. Al raggiungimento di 10 Gol, l'Edge dichiara la partita conclusa e pubblica il payload del risultato finale sul broker Cloud.

Postcondizioni:	La partita risulta conclusa in locale e i relativi dati sono registrati e persistenti sul Server Cloud tramite PostGreSQl.
Sequenze alternative:	A1 (Disconnessione Cloud), A2 (Dati Sensore Malformati o Incoerenti), A5 (Disconnessione Sensore LAN).

Caso d'uso: Gestione Partita Freccette e UI Locale

Dettaglio del caso uso 2	Calcolo punteggio e gestione tabellone Freccette
Id:	UC-02
Responsabile:	Timothy
Attori principali:	Giocatore (Fruitore UI), Simulatore Sensori
Attori secondari:	Edge Node, Server Cloud
Precondizioni:	1. Il simulatore delle Freccette è attivo in LAN. 2. Una partita è stata avviata e il punteggio iniziale è settato a 501.
Flusso principale:	1. Il sensore rileva l'impatto della freccetta e pubblica sul topic locale il settore colpito. 2. L'Edge Node riceve l'evento e calcola il punteggio sottratto applicando le logiche di gioco specifiche (es. chiusura in doppio).

	3. L'Edge aggiorna immediatamente il monitor della sala (JavaFX) per riflettere il punteggio rimanente del giocatore. 4. Quando il punteggio raggiunge lo 0 in modo valido, l'Edge chiude il match e inoltra il JSON di riepilogo al Server Cloud.
Postcondizioni:	Il giocatore visualizza la vittoria a schermo e la partita è salvata nel database globale.
Sequenze alternative:	A1 (Disconnessione Cloud), A2 (Dati Sensore Malformati o Incoerenti), A5 (Disconnessione Sensore LAN).

Caso d'uso: Sincronizzazione Dati Biliardo Offline/Online

Dettaglio del caso uso 3	Sincronizzazione differita eventi Biliardo
Id:	UC-03
Responsabile:	Luca
Attori principali:	Edge Node
Attori secondari:	Simulatore Sensori, Server Cloud
Precondizioni:	1. La connessione Internet del locale (verso il Cloud) è attualmente assente.

	2. Il client MQTT dell'Edge è configurato con la *Durable Session* attiva (setCleanSession(false)).
Flusso principale:	1. L'Edge rileva eventi di gioco dal Biliardo (es. "Palla 8 in buca") inviati dal sensore. 2. Il tentativo dell'Edge di pubblicare i dati aggiornati sul Cloud fallisce per timeout di rete. 3. Il gestore MQTT dell'Edge Node accoda il payload JSON internamente (in RAM o su Disco) mantenendo l'ordine cronologico. 4. La connessione di rete Internet torna disponibile. 5. La *Durable Session* si ripristina automaticamente col broker Cloud e l'Edge svuota la coda sincronizzando lo storico sul database centrale.
Postcondizioni:	Il database centrale Cloud e lo stato locale sono perfettamente allineati, con zero perdita di dati.
Sequenze alternative:	A1 (Disconnessione Cloud - costituisce la premessa di questo UC ma viene gestita qui intrinsecamente).

Caso d'uso: Consultazione Statistiche HATEOAS

Dettaglio del caso uso 4	Navigazione statistiche personali API REST
Id:	UC-04
Responsabile:	Timothy

Attori principali:	Giocatore (Client Web/App)
Attori secondari:	Server Cloud (HttpServer)
Precondizioni:	1. Il giocatore sta utilizzando un'interfaccia client (App o Web) connessa a Internet. 2. Il database centrale contiene uno storico di partite giocate associate a quell'utente.
Flusso principale:	1. Il client del giocatore esegue una richiesta HTTP GET mirata all'endpoint anagrafico (es. /api/v1/utenti/123). 2. L'HttpServer cloud elabora la richiesta associandola a uno dei thread del suo thread-pool. 3. Il server interroga in lettura PostgreSQL tramite il DAO e formula una risposta JSON state-less. 4. Il server include nel JSON i dati utente e una struttura _links (principio HATEOAS) contenente le URI dinamiche (es. /statistiche o /tornei-giocati). 5. Il client processa la risposta e utilizza direttamente l'URI fornita nel payload per effettuare la successiva transizione di stato (es. richiedere le statistiche).
Postcondizioni:	L'utente visualizza con successo le statistiche richieste navigando in modo disaccoppiato dall'albero URL statico del server.
Sequenze alternative:	A3 (Errore Concorrenza Database), A4 (Risorsa Non Trovata o Link HATEOAS Scaduto).

Caso d'uso: Creazione e Gestione Torneo (Singolo/Squadre)

Dettaglio del caso uso 5	Organizzazione competizione ludica nel locale
Id:	UC-05
Responsabile:	Stefano
Attori principali:	Gestore del Locale
Attori secondari:	Server Cloud
Precondizioni:	1. Il Gestore è autorizzato ad agire sul Locale di sua competenza. 2. Nel database centrale esistono tavoli fisici assegnati a quel locale (es. Calciobalilla 1).
Flusso principale:	1. Il Gestore, tramite interfaccia gestionale, invia un payload JSON tramite POST /api/v1/tornei con i parametri della competizione. 2. Il Server Cloud riceve la richiesta e valida i dati di business (es. raggiungimento del numero minimo di iscritti). 3. Il DAO thread-safe registra l'evento nel database SQLite e il server genera il tabellone degli scontri (bracket). 4. Il Server restituisce al Gestore un JSON di conferma (HATEOAS) contenente i link dedicati per la gestione dei singoli match del torneo appena creato.
Postcondizioni:	Il torneo è ufficialmente creato, associato al tavolo corretto, e i giocatori possono consultare gli accoppiamenti.

Sequenze alternative:	A3 (Errore Concorrenza Database), A4 (Risorsa Non Trovata o Link HATEOAS Scaduto).

Caso d'uso: Configurazione Anagrafica Piattaforma (CRUD)

Dettaglio del caso uso 6	Registrazione nuovi dispositivi Edge e Tavoli
Id:	UC-06
Responsabile:	Luca
Attori principali:	Amministratore di Sistema
Attori secondari:	Server Cloud
Precondizioni:	1. L'Amministratore dispone degli identificativi univoci hardware (UUID/MAC) del nuovo Edge Node e dei tavoli da registrare.
Flusso principale:	1. L'Amministratore usa l'interfaccia API REST (POST /api/v1/locali e POST /api/v1/giochi) inviando i dati anagrafici dei dispositivi da inserire. 2. Il DAO del Server Cloud intercetta la richiesta ed esegue controlli thread-safe per verificare che non esistano conflitti o duplicati (es. due tavoli con lo stesso UUID). 3. In assenza di conflitti, il server inserisce le entità nel database SQLite. 4. Il Server associa logicamente l'Edge Node e i tavoli all'entità "Locale" specificata.

Postcondizioni:	Il Server Centrale riconosce il nuovo hardware, rendendo il locale operativo e autorizzato a inviare dati MQTT sulle partite.
Sequenze alternative:	A3 (Errore Concorrenza Database).

6. Descrizione scenari alternativi (Estensioni ed Errori)

Scenario: Disconnessione Cloud

Dettaglio scenario	Scenario A1
Id:	A1
Breve descrizione:	Rischio critico mitigato dalla progettazione. La connessione tra l'Edge Node in LAN e il server TLS in Cloud si interrompe inaspettatamente durante lo svolgimento di una o più partite nel locale.
Attori Principali:	Edge Node, Server Cloud
Precondizioni:	L'Edge Node è funzionante, i sensori inviano dati, ma la rete Internet o il Server Cloud diventano improvvisamente irraggiungibili.
Flusso principale:	1. Il socket MQTT va in Timeout di rete.

	2. La libreria Eclipse Paho entra in stato disconnesso ma, avendo la *Durable Session* configurata (setCleanSession(false)), non scarta i messaggi in uscita (con QoS 1 o 2). 3. L'Edge Node accoda internamente i payload JSON contenenti gli eventi di gioco. 4. L'interfaccia JavaFX locale continua a funzionare fluidamente elaborando i logici di gioco e mostrando i punteggi aggiornati ai giocatori senza freeze della UI. 5. Una routine asincrona dell'Edge tenta automaticamente la riconnessione al broker Cloud a intervalli regolari (es. ogni *N* secondi).
Postcondizioni:	L'esperienza utente nel locale non subisce rallentamenti o disservizi. I dati sensibili restano accodati in sicurezza pronti per la sincronizzazione al ritorno della rete.

Scenario: Errore Concorrenza Database (SQLite Locked)

Dettaglio scenario	Scenario A3
Id:	A3
Breve descrizione:	Gestione della concorrenza del database centrale. Si verifica quando il Server Cloud riceve contemporaneamente molteplici messaggi di chiusura partita da diversi Edge Node sparsi sul territorio.

Attori Principali:	Server Cloud (HttpServer), DAO (Data Access Object)
Precondizioni:	Molteplici Edge Node ripristinano la connessione simultaneamente o inviano raffiche di dati contemporaneamente allo stesso Server Cloud.
Flusso principale:	1. Due o più thread dell'HttpServer tentano di eseguire query di scrittura (INSERT/UPDATE) simultaneamente sul DB SQLite (che nativamente soffre di colli di bottiglia in concorrenza elevata). 2. L'oggetto DAO preposto all'accesso dati intercetta le molteplici richieste. 3. I blocchi synchronized implementati nel DAO serializzano le code di scrittura, acquisendo un lock esclusivo sulla risorsa. 4. I thread concorrenti vengono messi in attesa sicura fino al rilascio del lock dal thread attivo. 5. La scrittura avviene in sequenza senza causare crash dell'applicazione.
Postcondizioni:	Tutti i dati provenienti dai locali vengono salvati in modo affidabile, garantendo la totale assenza di corruzione dei file e rispettando rigorosamente le proprietà ACID del database.

Scenario: Dati Sensore Malformati o Incoerenti (MQTT)

Dettaglio scenario	Scenario A2
Id:	A2
Breve descrizione:	Anomalie del traffico LAN. Il simulatore hardware genera un evento impossibile o il payload JSON arriva corrotto (es. il sensore delle freccette invia un settore "25", inesistente nel gioco reale).
Attori Principali:	Edge Node, Simulatore Sensori
Precondizioni:	Una partita è in corso e l'Edge Node è in ascolto attivo sugli eventi MQTT pubblicati dal simulatore sul broker locale.
Flusso principale:	1. Il simulatore pubblica il payload JSON contenente il dato errato sul topic MQTT. 2. L'Edge Node riceve il messaggio tramite la libreria Eclipse Paho. 3. La libreria GSON tenta il parsing dell'oggetto e il motore logico dell'Edge ne valida i vincoli di dominio.

	4. L'Edge rileva l'incoerenza logica o strutturale. 5. Il messaggio corrotto viene immediatamente scartato per evitare di inquinare lo stato della partita corrente. 6. L'Edge Node scrive un avviso (Warning) nel file di log locale per facilitare il debugging.
Postcondizioni:	La partita continua in modo fluido. L'interfaccia UI JavaFX non subisce variazioni errate e l'integrità del punteggio è preservata.

Scenario: Risorsa Non Trovata o Link HATEOAS Scaduto

Dettaglio scenario	Scenario A4
Id:	A4
Breve descrizione:	Gestione degli errori di navigazione REST. L'HttpServer state-less riceve una richiesta per un'entità che è stata rimossa o non è più disponibile nel sistema.
Attori Principali:	Giocatore (Client Web/App), Server Cloud
Precondizioni:	Un utente o un'applicazione client richiede una risorsa non più esistente nel database centrale (es. GET /api/v1/utenti/999 per un utente eliminato o un link HATEOAS obsoleto).
Flusso principale:	1. Il client effettua la richiesta HTTP GET seguendo un vecchio link HATEOAS o inserendo l'URL manualmente. 2. Il thread dell'HttpServer intercetta la richiesta ed esegue la query appropriata tramite il DAO. 3. Il database (SQLite) restituisce un ResultSet vuoto.

	4. Il controller dell'API rileva l'assenza della risorsa. 5. Il server formula una risposta rigorosa contenente un JSON d'errore standardizzato (con un messaggio descrittivo dell'accaduto). 6. La risposta viene inviata al client con lo status code HTTP *404 Not Found*.
Postcondizioni:	Il server gestisce l'eccezione elegantemente senza andare in crash. Il client riceve un feedback chiaro e standardizzato e può adattare la propria interfaccia di conseguenza.

Scenario: Disconnessione Sensore

Dettaglio scenario	Scenario A5
Id:	A5
Breve descrizione:	Guasto all'infrastruttura di rete locale. Un singolo tavolo da gioco (es. il Biliardo) perde la connessione MQTT verso l'Edge Node a causa di un cavo staccato, Wi-Fi instabile o perdita di alimentazione del sensore.
Attori Principali:	Edge Node, Simulatore Sensore, Broker MQTT Locale
Precondizioni:	Il tavolo da gioco è attivo e connesso al broker locale. Una partita è in corso.
Flusso principale:	1. Il sensore hardware/simulatore perde repentinamente la connessione con la rete LAN. 2. Il Broker Mosquitto locale rileva il mancato invio dei pacchetti di Keep-Alive e dichiara il timeout del client. 3. Il Broker pubblica automaticamente sul topic dedicato l'ultimo messaggio di emergenza configurato dal sensore alla connessione ("Last Will and Testament" - LWT).

	4. L'Edge Node riceve il messaggio LWT associato a quello specifico tavolo. 5. L'Edge Node interrompe e invalida la partita corrente a causa della perdita di affidabilità dei dati. 6. L'Edge aggiorna l'interfaccia JavaFX posizionando un overlay di "Tavolo in Manutenzione" per avvisare gli avventori.
Postcondizioni:	Il sistema previene la registrazione di partite parziali o fasulle. Il Gestore del locale e i giocatori sono immediatamente informati del disservizio hardware tramite il monitor.

7. Matrice di mappatura dei requisiti

Requisito \ Caso d'Uso	**UC-01 (Calciobalilla)**	**UC-02 (Freccette)**	**UC-03 (Biliardo / Sync)**	**UC-04 (Statistiche)**	**UC-05 (Tornei)**	**UC-06 (Anagrafica)**
RF-01 (Acquisizione Dati Sensoriali)	**X**	**X**	**X**			
RF-02 (Sincronizzazione Offline/Online)	**X**	**X**	**X**			

Requisito \ Caso d'Uso	**UC-01 (Calciobalilla)**	**UC-02 (Freccette)**	**UC-03 (Biliardo / Sync)**	**UC-04 (Statistiche)**	**UC-05 (Tornei)**	**UC-06 (Anagrafica)**
RF-03 (Esposizione API REST)				**X**	**X**	**X**
RF-04 (Gestione Tornei e Squadre)					**X**	
RF-05 (Elaborazione Statistiche)	**X**	**X**	**X**	**X**		
RF-06 (Interfaccia Utente Locale)	**X**	**X**	**X**			
RF-07 (Gestione Anagrafiche)						**X**